

Chapter 10: Multi-Agent Setting

An Introduction to Universal Artificial Intelligence

Utilities, Game Theory, Reflective Oracles, and the Grain of Truth

Marcus Hutter, Elliot Catt, David Quarel

Taylor & Francis / CRC Press, 2024

Chapter Overview

- 1 10.1 From Preferences to Utilities
- 2 10.2 Game Theory
- 3 10.3 Multi-Agent Extensive-Form Games
- 4 10.4 Strategic Games vs Reinforcement Learning
- 5 10.5 Reflective Oracles
- 6 10.6 The Grain of Truth
- 7 10.7 Reflective AIXI
- 8 10.8 Exercises
- 9 10.9 History and References

Outline

- 1 10.1 From Preferences to Utilities
- 2 10.2 Game Theory
- 3 10.3 Multi-Agent Extensive-Form Games
- 4 10.4 Strategic Games vs Reinforcement Learning
- 5 10.5 Reflective Oracles
- 6 10.6 The Grain of Truth
- 7 10.7 Reflective AIXI
- 8 10.8 Exercises
- 9 10.9 History and References

Why a Multi-Agent Chapter?

So far this book has only discussed the **single-agent** setting, where there is one agent and the environment. More realistically, and more complex, is the **multi-agent setting**, where we consider the agent, the environment, *and* some number of other agents.

Why not just fold the other agents into the environment? We could try to capture the behaviour of other agents as part of the environment. While this is technically possible, it misses some of the subtlety and advantages of explicitly considering the multi-agent setting: when we *know* we are facing other agents, we know they will also learn, or at least act in specific, structured ways with respect to our own actions – and we can try to exploit that structure.

Roadmap for this chapter.

- **10.1** A classic decision-theory result: rational preferences can be represented by a real-valued (von Neumann–Morgenstern) *utility function* – the basis of expected utility theory and hence most of game theory.
- **10.2** The basics of **game theory**: strategic games, Nash equilibrium, and five famous 2×2 games.
- **10.3–10.4** How multi-agent *extensive-form* games relate to (history-based) reinforcement learning.
- **10.5–10.7** **Reflective oracles**, the **grain of truth** problem, and **Reflective AIXI** – how to let agents reason about opponents that may themselves be reasoning about them, without an infinite regress.

Preferences and Lotteries

We would like to formalize what it means for an agent to be **rational**. Let $\mathcal{O}$ (calligraphic $\mathcal{O}$) denote a set of **outcomes** (this could even be the set of all finite interaction histories), and let $\prec$ (“precedes”, pronounced “is preferred less than”) denote the agent’s **preference**, a binary relation over $\mathcal{O}$.

In practice, a full description of the preferences $\prec$ of an agent (like a human) may in general be unknown. We treat the agent’s preferences as a black box, and assume we can *query* the agent: if the agent strictly prefers a to b we write $a \succ b$, or $b \prec a$. If the agent is indifferent between a and b (formally, $a \not\succ b$ and $b \not\succ a$) we write $a \sim b$. The symbols $\succeq$ and $\preceq$ (“at least as preferred as” / “no more preferred than”) are defined in the obvious fashion.

Lotteries. Given two outcomes $a, b \in \mathcal{O}$ and θ (Greek letter *theta*) $\in [0, 1]$, we define a **lottery** $\theta a + (1 - \theta)b$ as the random outcome representing: observe outcome a with probability θ , and outcome b with probability $1 - \theta$. We require that $\mathcal{O}$ is *closed under lotteries*, and that the agent also has preferences defined for lotteries – that is, given any outcomes $a, a', b, b' \in \mathcal{O}$ and $\alpha, \beta \in [0, 1]$ (α = “alpha”, β = “beta”), the outcomes $\alpha a + (1 - \alpha)a'$ and $\beta b + (1 - \beta)b'$ are themselves in $\mathcal{O}$, and hence comparable with respect to $\prec$. This is what makes outcomes *random events* in general.

Definition 10.1.1: Utilitarian Preferences

Definition 10.1.1 (Utilitarian preferences)

An agent's preferences $\prec$ are *utilitarian* if there exists a function $U : \mathcal{O} \rightarrow \mathbb{R}$ such that

$$a \prec b \quad \text{if and only if} \quad \mathbb{E}[U(a)] < \mathbb{E}[U(b)]$$

Plain English. U is a **utility function** that assigns a single real number (a “score”) to every outcome. The preference relation $\prec$ is called utilitarian if we can find some scoring function U such that ranking outcomes by their *expected* score $\mathbb{E}[U(\cdot)]$ (the average score, over the randomness of the outcome when it is a lottery) exactly reproduces the agent's preferences: a is preferred less than b precisely when a 's expected score is lower than b 's. If such a U exists, we can reduce a comparison between two outcomes to comparing two numbers – much more convenient than reasoning about an abstract preference relation directly.

Theorem 10.1.2: The von Neumann–Morgenstern Utility Theorem

Theorem 10.1.2 (von Neumann–Morgenstern utility [VNM47])

Let $\prec$ be a preference relation over outcomes $\mathcal{O}$. If the four following conditions hold for arbitrary $a, b, c \in \mathcal{O}$:

- ① **Completeness:** At least one of the following is true: $a \preceq b$ or $b \preceq a$.
- ② **Transitivity:** If $a \preceq b$ and $b \preceq c$, then $a \preceq c$.
- ③ **Continuity:** If $a \preceq b \preceq c$ then there exists $\theta \in [0, 1]$ such that $\theta a + (1 - \theta)c \sim b$.
- ④ **Independence:** $a \preceq b$ if and only if $\theta a + (1 - \theta)c \preceq \theta b + (1 - \theta)c$ for any $\theta \in (0, 1)$.

then $\prec$ is utilitarian.

Plain English. If an agent's preferences satisfy these four intuitively reasonable axioms, then the theorem *guarantees* a utility function U exists (Definition 10.1.1) – we do not need to construct it by hand. This is a foundational result: it is a partial justification for the *reward hypothesis* used since Chapter 6, and the basis of expected utility theory (hence most of game theory).

Understanding the Four VNM Conditions I

The theorem is not proved in the book; instead, here is why each condition is reasonable.

- ① **Completeness.** This merely assumes that between any two options, the agent can state which one it prefers (or state that it is indifferent). If the agent cannot even state a preference between two options, we have no hope of assigning utilities to it.
- ② **Transitivity.** This seems intuitively obvious (if the agent prefers apples to bananas, and bananas to pears, it would stand to reason the agent prefers apples to pears). Suppose $\preceq$ were non-transitive: there exist a, b, c such that $a \preceq b$ and $b \preceq c$ but $a \succ c$. Then, presumably an agent would be happy (or indifferent) to swap a for b , and the same for swapping b for c , but then the agent *strictly* prefers a to c , and would pay a non-zero amount (say, one cent) to swap c back for a . The agent is now back in the initial configuration, and one penny poorer. We can repeat this argument *ad infinitum* and take all of the agent's assets, one penny at a time. This style of argument, used to demonstrate that non-transitive preferences are irrational, is called a **money pump**.

Understanding the Four VNM Conditions II

- ③ **Continuity.** This requires that the preferences of an agent do not change suddenly in response to arbitrarily small changes. If the agent prefers $a \preceq b \preceq c$, then the lottery $I = \theta a + (1 - \theta)c$ should still satisfy $a \preceq I \preceq b$ for $\theta \approx 1$ (since I would with overwhelming probability be the same outcome as a , and $a \preceq b$). As θ is decreased, the agent would prefer I more and more (as the outcome is more likely to be the more desirable outcome c), and at some point I is almost indistinguishable from c for $\theta \approx 0$, so we would expect $b \preceq I \preceq c$. It is reasonable to assume that for some intermediate value of θ , the agent is indifferent between I and b .
- ④ **Independence.** Given a preference $a \preceq b$, the agent should still prefer a θ chance at b over a θ chance at a , regardless of the other outcome c with $1 - \theta$ probability shared by both lotteries. This indicates that a preference should not change if stated as a lottery, as long as the alternative option is the same for both lotteries being compared.

Theorem 10.1.2 allows us to work with real-valued *utilities* rather than abstract preference relations, which is far more convenient – and it provides a partial justification for the reward hypothesis we assumed since Chapter 6.

Outline

- 1 10.1 From Preferences to Utilities
- 2 10.2 Game Theory
- 3 10.3 Multi-Agent Extensive-Form Games
- 4 10.4 Strategic Games vs Reinforcement Learning
- 5 10.5 Reflective Oracles
- 6 10.6 The Grain of Truth
- 7 10.7 Reflective AIXI
- 8 10.8 Exercises
- 9 10.9 History and References

What Is Game Theory?

Game theory is the study of multi-agent interactions. For this section we use the notation of [OR94]; for a more in-depth treatment see [Osb04, SLB08].

10.2.1 Strategic games. The common definition of a *game* is a 3-tuple with players, actions, and preferences. This is called a **strategic game** (in normal form).

Definition 10.2.1 (Strategic game in normal form)

A *strategic game* is a 3-tuple $\langle N, (\mathcal{A}_i)_{i \in N}, (\succeq_i)_{i \in N} \rangle$, where N is a finite set of **players**, $\mathcal{A}_i$ is the set of **actions** for player i , and $\succeq_i$ is the **preference relation** on $\mathcal{A} := \mathcal{A}_1 \times \mathcal{A}_2 \times \dots \times \mathcal{A}_N$ for player i . If all of the action sets $\mathcal{A}_i$ are finite, we call the 3-tuple a *finite strategic game*.

Action Profiles and Utility Functions

Given a subset $I \subset N$ of the players, an assignment of players to actions $f : I \rightarrow \mathcal{A}_i$ is called an **action profile**, or simply a *profile*. We usually denote an action profile as a vector $a \in \prod_{i \in I} \mathcal{A}_i$ of each player's action.

We assume that for each player i there is a **utility function** $u_i : \mathcal{A} \rightarrow \mathbb{R}$ such that for all $a, b \in \mathcal{A}$, $u_i(b) \geq u_i(a)$ if and only if $b \succeq_i a$. This implicitly places constraints on the preferences players are allowed to have – it forbids preferences with inconsistencies that prevent them from being modelled as the maximization of some utility function, as discussed in Section 10.1. Where preferences can be specified by a utility function, we skip preferences entirely and define $\langle N, (\mathcal{A}_i)_{i \in N}, (u_i)_{i \in N} \rangle$ as the strategic game. We usually present strategic games this way, since non-utilitarian preferences are irrational (Theorem 10.1.2).

The $a_{\neq i}$ notation. For any profile $a = (a_1, \dots, a_n) \in \mathcal{A}$, denote by $a_{\neq i}$ the vector $(a_1, \dots, a_{i-1}, a_{i+1}, \dots, a_n)$: the *subprofile* of a with the action associated with player i removed. As a slight abuse of notation, we write $u_j(a_{\neq j}, b)$ for player j 's utility for taking action b instead of action a_j , assuming the actions of all other players are described by the action profile $a_{\neq j}$.

10.2.2 Nash Equilibrium

Informally, a **Nash equilibrium** is an action profile with the property that no player would prefer to unilaterally change their action, given knowledge of the actions that all other players will choose.

Definition 10.2.2 (Nash equilibrium)

A *Nash equilibrium* of a strategic game $\langle N, (\mathcal{A}_i)_{i \in N}, (\succeq_i)_{i \in N} \rangle$ is an action profile $a^* \in \mathcal{A}$ with the property that for every player $i \in N$,

$$\forall a \in \mathcal{A}_i. u_i(a^*) \geq u_i(a_{\neq i}^*, a)$$

Plain English. That is, given every other player chooses the action specified in the profile a^* , there is no player i who would (strictly) prefer to play some action other than a_i^* : every player is already doing the best they can, given what everyone else is doing.

Definition 10.2.3 (Best response function)

Given $a_{\neq i}$, the action profile of all other players, we define player i 's *best response function* $B_i : (\mathcal{A}_j)_{j \in N \setminus i} \rightarrow \mathcal{A}_i$ as the set of all actions that maximize their utility,

$$B_i(a_{\neq i}) := \arg \max_{a \in \mathcal{A}_i} u_i(a_{\neq i}, a)$$

Theorem 10.2.4 and the General Payoff Matrix

Theorem 10.2.4 (Nash equilibrium via best response functions)

An action profile a^* is a Nash equilibrium if and only if for all i , every action a_i^* is one of player i 's best responses to $a_{\neq i}^*$, that is, $a_i^* \in B_i(a_{\neq i}^*)$.

Finite two-player normal-form strategic games are often represented as a **payoff matrix**: rows are the actions of one player, columns are the actions of the other. Each cell holds a pair (u_1, u_2) , where u_1 is the utility for the row player (Player 1) and u_2 for the column player (Player 2), for that action profile.

	<i>action 0</i>	<i>action 1</i>
<i>action 0</i>	$(u_1(0, 0), u_2(0, 0))$	$(u_1(0, 1), u_2(0, 1))$
<i>action 1</i>	$(u_1(1, 0), u_2(1, 0))$	$(u_1(1, 1), u_2(1, 1))$

It is not immediately clear whether a Nash equilibrium exists for every strategic game – and indeed, often it does not.

Are Nash Equilibria Optimal?

The surprising, and often frustrating, property of some games is that a Nash equilibrium a^* is *not* the optimal profile for all players: there can exist a non-Nash profile b^* that all players would strictly prefer ($\forall i. u_i(b^*) > u_i(a^*)$).

Unfortunately, non-Nash-equilibrium strategies are *not stable*, in the sense that there exists a player i who would gain even more utility by unilaterally choosing a different action $a_i \neq b_i^*$ – often at the expense of decreasing the utility of the other players.¹ After several iterations of the same game, other players might act “selfishly” in the same manner, until the action profile for which no player would unilaterally change their action is reached – which is strictly worse off for everyone. A classical example is the **Prisoner’s Dilemma**, described later in this section.

¹Assuming each agent is *egotistical*, valuing only outcomes for themselves, as opposed to *altruistic* agents who value the utility of both themselves and others.

Guaranteeing Existence: Kakutani's Fixed Point Theorem

If a strategic game satisfies certain conditions, existence of a Nash equilibrium is guaranteed. The key mathematical tool is a fixed-point theorem.

Lemma 10.2.5 (Kakutani's fixed point theorem [Kak41])

Let X be a compact convex subset of $\mathbb{R}^n$ and let $f : X \rightarrow 2^X$ be a set-valued function for which:

- 1 For all $x \in X$, the set $f(x)$ is nonempty and convex, i.e. for all $x_1, x_2 \in f(x)$ the line segment from x_1 to x_2 is contained in $f(x)$, i.e. for all $\alpha \in [0, 1]$, $\alpha x_1 + (1 - \alpha)x_2 \in f(x)$.
- 2 For all sequences (x_n) and (y_n) such that for all n , $y_n \in f(x_n)$, and the sequences have limits $x_n \rightarrow x$ and $y_n \rightarrow y$, then $y \in f(x)$ – i.e. the graph of f contains its limit points (the graph is *closed*).

Then there exists a fixed point x^* such that $x^* \in f(x^*)$.

Plain English. If a set-valued map on a nice (compact, convex) space always outputs nonempty, convex, “well-behaved” (closed-graph) sets, some point must map to a set containing itself.

Theorem 10.2.7: Existence of Nash Equilibrium I

Definition 10.2.6 (Quasi-concave preference relation)

A preference relation $\succeq_i$ over $\mathcal{A}$ is *quasi-concave* over $\mathcal{A}_i$ if for every $a \in \mathcal{A}$, the set $\{a_i \in \mathcal{A}_i : (a_{\neq i}, a_i) \succeq_i a\}$ is convex.

Plain English. This says: for player i , fixing everyone else's actions, the set of i 's own actions that are “at least as good” as some baseline is a convex set – no “holes” or disconnected islands of good actions.

Theorem 10.2.7 (Existence of Nash equilibrium [OR94, Prop. 20.3])

The strategic game $\langle N, (\mathcal{A}_i)_{i \in N}, (\succeq_i)_{i \in N} \rangle$ has a Nash equilibrium if for all $i \in N$:

- 1 The set $\mathcal{A}_i$ is a nonempty, compact, convex subset of $\mathbb{R}^n$.
- 2 The preference relation $\succeq_i$ is continuous and quasi-concave on $\mathcal{A}_i$.

Theorem 10.2.7: Existence of Nash Equilibrium II

Proof sketch. For $B : \mathcal{A} \rightarrow \mathcal{A}$ defined as $B(a) := \times_{i \in N} B_i(a_{\neq i})$, the condition for a being a Nash equilibrium in Theorem 10.2.4 can be rewritten as $B(a^*) = a^*$. For every $i \in N$ the set $B_i(a_{\neq i})$ is nonempty since $\succeq_i$ is continuous and $\mathcal{A}_i$ is compact, and is convex since $\succeq_i$ is quasi-concave on $\mathcal{A}_i$; B has a closed graph since each $\succeq_i$ is continuous. Thus by Kakutani's theorem, B has a fixed point, and hence by Theorem 10.2.4 the above is a Nash equilibrium. ■

Definition 10.2.8 (Strictly competitive game)

A strategic game $(\{1, 2\}, (\mathcal{A}_1, \mathcal{A}_2), (u_1, u_2))$ with two players is a *competitive game* if for all action profiles $a, b \in \mathcal{A}$, $a \succeq_1 b$ if and only if $b \succeq_2 a$.

Plain English. One player's gain is exactly the other's loss. One can show that for strictly competitive games, there are *zero-sum* payoff functions ($u_1 + u_2 = 0$) representing the players' preferences.

10.2.3 Important Games

In game theory, many interesting normal-form games have been studied. We now go over **five** prototypical 2×2 normal-form games, which we later use as environments for a simple approximation of AIXI. These games have been intensely studied since they are the smallest non-trivial games, already exhibiting a wide variety of interesting and intricate phenomena.

Prisoner's Dilemma

It would not be an exaggeration to say that the **Prisoner's Dilemma** is the most studied game in game theory. It is a fundamental game for which the Nash equilibrium is strictly suboptimal for both players. Two suspects of a crime are interrogated separately; each can *cooperate* with the other criminal (remain silent) or *defect* (testify against their co-conspirator). If both cooperate, they get a short sentence (utility 2 each). If both defect, they get a longer sentence (utility 1 each). If one defects while the other cooperates, the defector is released (utility 4) while the cooperator gets the longest sentence (utility 0).

		Player 2	
		<i>defect</i>	<i>cooperate</i>
Player 1	<i>defect</i>	(1, 1)	(4, 0)
	<i>cooperate</i>	(0, 4)	(2, 2)

Prisoner's Dilemma: The Nash Equilibrium

Ideally both players would wish to cooperate, but this creates an incentive for either player to defect: if the other player cooperates, defecting yields utility 4 instead of 2; if the other player defects, defecting still yields utility 1 instead of 0. Defecting is therefore the best response to *either* of the opponent's actions, leading to the Nash equilibrium of (defect, defect) – a strictly worse outcome for both players than (cooperate, cooperate).

Very serious real-world versions of this dilemma are the *tragedy of the commons* for shared physical resources, which (without global cooperation) leads to pollution, overfishing, and deforestation.

Stag Hunt

Two hunters are on a hunting trip together. They may cooperate to hunt a *stag*, or hunt *alone* for a rabbit. If both hunt alone, they each catch and share a rabbit (utility 2). If they hunt together, they catch a stag and both obtain utility 4. If one hunts alone while the other hunts the stag, the rabbit hunter gets the rabbit to himself (utility 3), while the lone stag-hunter is unsuccessful (utility 0).

		Player 2	
		<i>alone</i>	<i>together</i>
Player 1	<i>alone</i>	(2, 2)	(3, 0)
	<i>together</i>	(0, 3)	(4, 4)

Stag Hunt: Cooperation Is Also a Nash Equilibrium

Stag Hunt may look similar to the Prisoner's Dilemma, but has one key difference: the *highest* utility for each player, (together, together), is *also* a Nash equilibrium ((alone, alone) is a second, worse, Nash equilibrium). This means the players have no incentive to fail to work together, and it is in the best interest of even selfish players to cooperate.

Other examples with the same structure: two individuals rowing a boat, neighbours draining a meadow, coordination of slime molds, hunting practices of orcas, and countries improving corporate governance together.

Chicken

Two thrill-seekers play a game of brinkmanship: both drive cars toward each other on a head-on collision course. Either driver can *swerve* or *not swerve*. If one swerves and the other does not, the swerver is a “chicken” (utility 1) while the other gains admiration (utility 4). If both swerve, neither is branded a coward but neither gets admiration either (utility 2 each). If neither swerves, they crash head-on and receive utility 0 each.

		Player 2	
		<i>no swerve</i>	<i>swerve</i>
Player 1	<i>no swerve</i>	(0, 0)	(4, 1)
	<i>swerve</i>	(1, 4)	(2, 2)

Chicken: Two Nash Equilibria

Unlike the previous two games, Chicken has **two** Nash equilibria: (no swerve, swerve) and (swerve, no swerve). In either case, the chickening player switching to not-swerving gets 0 instead of 1, and the non-swerving player switching to swerving gets 2 instead of 4 – neither wants to unilaterally deviate.

Chicken is popular in politics, economics, and military affairs (achieving an advantage by pushing dangerous events to the brink), price wars between companies, or two people over-claiming credit for a joint project.

Battle of the Sexes

A couple intends to spend an evening together. The husband prefers a *Western*, the wife prefers a *Musical*, but both would rather go to the *same* event than different ones. Each receives utility 4 if they attend their preferred event together, utility 2 if they attend the event they dislike together with their spouse, and utility 0 if they go separately (regardless of choice).

		Husband	
		<i>Musical</i>	<i>Western</i>
Wife	<i>Musical</i>	(2, 4)	(0, 0)
	<i>Western</i>	(0, 0)	(4, 2)

Battle of the Sexes: Two Nash Equilibria

Similar to Chicken, Battle of the Sexes has two Nash equilibria: (Musical, Musical) and (Western, Western). However, unlike Chicken, if the opponent changes their action here, that will *not* benefit you – both equilibria are “coordination” outcomes, not conflicts.

In repeated play, couples may alternate between the Musical and the Western – an average utility of 3 per game, as each player gets utility 4 half the time and utility 2 half the time (a real-life “compromise”). Setting industry standards is another instance of Battle of the Sexes: each company prefers its in-house solution, but both are better off agreeing on a common standard.

Matching Pennies

Two players each choose *Heads* or *Tails*. Player 1 receives utility 1 if both choose the *same* option (0 otherwise). Player 2 receives utility 1 if they choose *different* options (0 otherwise).

		Player 2	
		Heads	Tails
Player 1	Heads	(1, 0)	(0, 1)
	Tails	(0, 1)	(1, 0)

Matching Pennies: No Pure Nash Equilibrium

Unlike all the previous games, Matching Pennies has **no pure (deterministic) Nash equilibrium**: for every action profile, the “losing” player can gain more utility by switching her strategy to the other action – there is always someone who wants to deviate.

Penalty kicks in soccer are an instance of Matching Pennies (kicker: left/right; goalie: jump left/right, before seeing the kick), as are serve-and-return plays in tennis. Rock-Paper-Scissors can be regarded as a three-action version of Matching Pennies.

Python: Finding Pure Nash Equilibria by Brute Force

We encode each game as a payoff dictionary and brute-force search all 4 action profiles for Nash equilibria (Theorem 10.2.4).

```
1 games = {
2     "PD": {(0,0):(1,1), (0,1):(4,0), (1,0):(0,4), (1,1):(2,2)},
3     "SH": {(0,0):(2,2), (0,1):(3,0), (1,0):(0,3), (1,1):(4,4)},
4     "Chicken": {(0,0):(0,0), (0,1):(4,1), (1,0):(1,4), (1,1):(2,2)},
5     "BoS": {(0,0):(2,4), (0,1):(0,0), (1,0):(0,0), (1,1):(4,2)},
6     "MP": {(0,0):(1,0), (0,1):(0,1), (1,0):(0,1), (1,1):(1,0)},
7 }
8 def is_nash(payload, r, c):
9     u1, u2 = payload[(r, c)]
10    if payload[(1 - r, c)][0] > u1: return False    # row player deviates
11    if payload[(r, 1 - c)][1] > u2: return False    # column player deviates
12    return True
13 for name, payload in games.items():
14     nash = [(r, c) for r in (0, 1) for c in (0, 1) if is_nash(payload, r, c)]
15     print(f"{name:9s} pure Nash equilibria: {nash}")
16 # PD          pure Nash equilibria: [(0, 0)]
17 # SH          pure Nash equilibria: [(0, 0), (1, 1)]
18 # Chicken     pure Nash equilibria: [(0, 1), (1, 0)]
19 # BoS         pure Nash equilibria: [(0, 0), (1, 1)]
20 # MP         pure Nash equilibria: []
```

This matches Theorem 10.2.4 exactly, and confirms Matching Pennies has no pure Nash equilibrium.

10.2.4 Mixed Strategic Games

When acting against intelligent players in a game (especially for competitive games, where cooperation is not an option), we do not want to always deterministically choose an action, lest our opponent predicts our action (as in Matching Pennies, where a predictable player always loses). This leads us to **mixed strategic games**: instead of choosing an action deterministically, each player chooses a **probability distribution** over the set of actions, then samples an action according to that distribution.

Definition 10.2.9 (Mixed strategic games in normal form)

The *mixed extension* of the strategic game $\langle N, (\mathcal{A}_i), (u_i) \rangle$ is the strategic game $\langle N, (\Delta\mathcal{A}_i), (U_i) \rangle$, where $\Delta\mathcal{A}_i$ is the set of all probability distributions over $\mathcal{A}_i$, and $U_i : \times_{j \in N} \Delta\mathcal{A}_j \rightarrow \mathbb{R}$ is defined as the expected value of u_i , assuming player i samples his action from $\mathcal{A}_i$ according to the distribution α_i . Writing $\alpha := (\alpha_1, \dots, \alpha_N)$, we can express U_i as

$$U_i(\alpha) := \sum_{a \in \mathcal{A}} u_i(a) \prod_{j \in N} \alpha_j(a_j)$$

Plain English. $U_i(\alpha)$ is the average payoff to player i when every player independently samples an action from their own distribution α_j ; $\prod_{j \in N} \alpha_j(a_j)$ is the probability of the specific joint profile a occurring under independent sampling.

Definition 10.2.10 and Theorem 10.2.11: Mixed Nash

We call a distribution in $\Delta\mathcal{A}_i$ a **mixed strategy** for player i . A *pure strategy* is the special case of a mixed strategy that is deterministic (assigns probability 1 to one action, 0 to the rest).

Definition 10.2.10 (Mixed Nash equilibrium)

A *mixed strategy Nash equilibrium* of a strategic game is a Nash equilibrium of the mixed extension: a collection $\alpha^* = (\alpha_1^*, \dots, \alpha_N^*)$ such that each $\alpha_i^* \in \Delta\mathcal{A}_i$, and no player would gain more (expected) utility by choosing a different distribution: for all players i and all $\alpha_i \in \Delta\mathcal{A}_i$,

$$U_i(\alpha^*) \geq U_i(\alpha_{\neq i}^*, \alpha_i)$$

Every ordinary (pure) Nash equilibrium a^* is a special case of a mixed Nash equilibrium, obtained via $\alpha_i^*(x) = \mathbb{I}[x = a_i^*]$.

Theorem 10.2.11 (Existence of mixed Nash equilibrium)

Every finite strategic game has a mixed strategy Nash equilibrium.

Proof sketch. Identify $\Delta\mathcal{A}_i$ with the probability simplex $\{(p_1, \dots, p_{m_i}) : p_j \geq 0, \sum_j p_j = 1\}$, where $m_i := |\mathcal{A}_i|$. This set is nonempty, convex, and compact. Since expectations are linear, one can define $a \succeq_i b$ iff $U_i(a) \geq U_i(b)$, which satisfies the premises of Theorem 10.2.7 for the mixed extension. ■

Finding the Mixed Nash Equilibrium of Matching Pennies

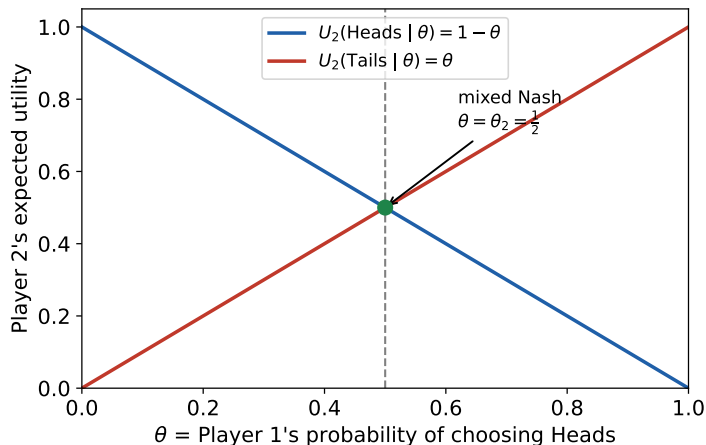
Recall that Matching Pennies has *no* pure Nash equilibrium. However, since the game is finite, Theorem 10.2.11 guarantees a mixed Nash equilibrium exists.

If Player 1 chooses (Heads, Tails) with probabilities $(\frac{1}{2}, \frac{1}{2})$, then the expected utility for both players is $\frac{1}{2}$, regardless of what strategy the opponent chooses – mixed or otherwise. Hence, if both players choose the mixed strategy $(\frac{1}{2}, \frac{1}{2})$, neither has an incentive to switch to any other mixed strategy (all mixtures are equally good against a uniformly random opponent), so both players choosing $(\frac{1}{2}, \frac{1}{2})$ is a mixed Nash equilibrium.

In fact, it is the *only* such equilibrium. If Player 1 chose some other mixed strategy $(\theta, 1 - \theta)$ with $\theta \neq \frac{1}{2}$, then Player 2 would choose a deterministic strategy of always Heads ($\theta < \frac{1}{2}$) or always Tails ($\theta > \frac{1}{2}$) – giving Player 2 expected utility $\max\{\theta, 1 - \theta\} > \frac{1}{2}$. The same argument applies to Player 1 if Player 2 deviates from $(\frac{1}{2}, \frac{1}{2})$.

Matching Pennies: Visualizing the Mixed Nash Equilibrium

The plot below shows Player 2's expected utility from playing Heads versus Tails, as a function of θ (Player 1's probability of playing Heads). The two lines cross exactly at $\theta = \frac{1}{2}$, which is where neither of Player 2's pure actions is strictly better than the other – confirming that $(\frac{1}{2}, \frac{1}{2})$ is the mixed Nash equilibrium.



Python: Verifying the Mixed Nash Equilibrium of Matching Pennies

We numerically check that Player 2's expected utility is maximized at $\theta = \frac{1}{2}$ (Player 1's probability of Heads), confirming no profitable deviation exists at the symmetric mixture.

```
1 def u2_expected(theta, p2_heads):
2     # Player 2 gets utility 1 iff actions differ.
3     # theta = P(P1=Heads); p2_heads = P(P2=Heads)
4     p_diff = theta * (1 - p2_heads) + (1 - theta) * p2_heads
5     return p_diff
6
7 theta = 0.5
8 for p2_heads in [0.0, 0.25, 0.5, 0.75, 1.0]:
9     print(f"P2 plays Heads w.p. {p2_heads:.2f} -> "
10         f"U2 = {u2_expected(theta, p2_heads):.3f}")
11
12 # P2 plays Heads w.p. 0.00 -> U2 = 0.500
13 # P2 plays Heads w.p. 0.25 -> U2 = 0.500
14 # P2 plays Heads w.p. 0.50 -> U2 = 0.500      (all mixtures tie: theta=1/2 is Nash)
15 # P2 plays Heads w.p. 0.75 -> U2 = 0.500
16 # P2 plays Heads w.p. 1.00 -> U2 = 0.500
17
18 theta = 0.7    # if Player 1 deviates from 1/2 ...
19 best = max(u2_expected(theta, p) for p in [0.0, 1.0])
20 print(f"theta=0.7: best pure response for P2 gives U2={best:.3f} > 0.5")
21 # theta=0.7: best pure response for P2 gives U2=0.700 > 0.5    (P2 deviates!)
```

Outline

- 1 10.1 From Preferences to Utilities
- 2 10.2 Game Theory
- 3 10.3 Multi-Agent Extensive-Form Games**
- 4 10.4 Strategic Games vs Reinforcement Learning
- 5 10.5 Reflective Oracles
- 6 10.6 The Grain of Truth
- 7 10.7 Reflective AIXI
- 8 10.8 Exercises
- 9 10.9 History and References

From One-Shot to Multi-Round Games

As currently defined, a strategic game has a *single round* of interaction. More interesting are games where all players choose an action (or an action is sampled from a chosen distribution), the players' actions are revealed, players receive utility according to the utility functions, and then players choose an action again for the next round, using the previous history of action profiles and received utilities as a guide. Multi-round strategic games are called **extensive-form games**.

This setting is extremely general – much more general than repeated (i.i.d.) matrix games. It generalizes the history-based (essentially assumption-free) reinforcement learning setting to modelling multiple agents, with simultaneous or sequential actions.

Figure 10.1: Agents in a Multi-Agent Environment

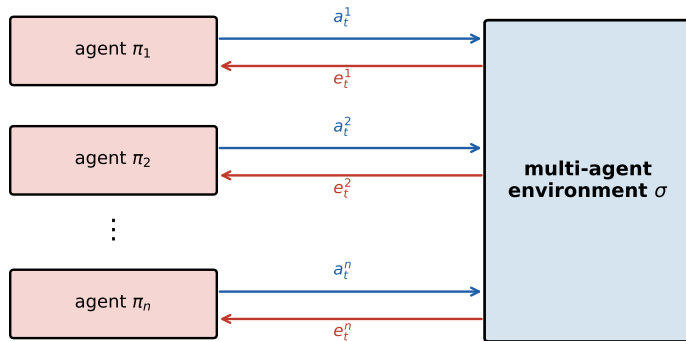


Figure 10.1. Agents $\pi_1, \dots, \pi_n$ (each π_i , “pi sub i ”, is agent i ’s policy) interact with a shared multi-agent environment σ (“sigma”), each only seeing its *own* action a_t^i and *own* percept e_t^i at every time step t .

Formalizing the Multi-Agent Environment

In an extensive-form game / multi-agent setting there are n agents, each taking actions sequentially from the finite action space $\mathcal{A}$. At each time step $t = 1, 2, \dots$, the environment receives an action a_t^i from agent i and outputs n percepts $e_t^1, \dots, e_t^n \in \mathcal{E}$, one for each agent. Each percept $e_t^i = (o_t^i, r_t^i)$ contains an observation o_t^i and a reward $r_t^i \in [0, 1]$. Importantly, agent i only sees its own action a_t^i and its own percept e_t^i (Figure 10.1).

Shorthand notation. We write $a_t := (a_t^1, \dots, a_t^n)$ and $e_t := (e_t^1, \dots, e_t^n)$ for the joint action/percept at time t , and denote $h_{<t}^i = a_1^i e_1^i \dots a_{t-1}^i e_{t-1}^i$ (agent i 's own history) and $h_{<t} = a_1 e_1 \dots a_{t-1} e_{t-1}$ (the full joint history).

We define a **multi-agent environment** as a probability kernel

$$\sigma : (\mathcal{A}^n \times \mathcal{E}^n)^* \times \mathcal{A}^n \rightarrow \Delta(\mathcal{E}^n)$$

Plain English. Given the full history so far and the joint action just taken by all n agents, σ returns a probability distribution over the joint percept $(e_t^1, \dots, e_t^n)$ that all agents will receive next.

The History Distribution

The n agents are given by n policies $\pi_1, \dots, \pi_n$, where $\pi_i : (\mathcal{A} \times \mathcal{E})^* \rightarrow \Delta\mathcal{A}$ (each agent's policy maps its own history to a distribution over its next action). Together, the policies and the environment specify the **history distribution**

$$\begin{aligned}\sigma^{\pi_{1:n}}(\epsilon) &:= 1 \\ \sigma^{\pi_{1:n}}(h_{1:t}) &:= \sigma^{\pi_{1:n}}(h_{<t}a_t) \sigma(e_t \mid h_{<t}a_t) \\ \sigma^{\pi_{1:n}}(h_{<t}a_t) &:= \sigma^{\pi_{1:n}}(h_{<t}) \prod_{i=1}^n \pi_i(a_t^i \mid h_{<t}^i)\end{aligned}$$

Plain English. The probability of the empty history is 1 (line 1). The probability of a history extended by one more joint percept e_t (line 2) is the probability of everything up to and including the joint action a_t , times the environment's chance of producing e_t given that action. The probability of extending a history by a joint action a_t (line 3) is the probability of the history so far, times the product over all n agents of the chance *that agent* takes the action it took, based only on *its own* history $h_{<t}^i$ – capturing that each agent decides independently, using only what it personally has observed.

Definition 10.3.1: Multi-Agent ε -Best Response

Our definition of a multi-agent environment is very general and encompasses most of game theory: it allows for cooperative, competitive, and mixed games; infinitely repeated games; or any (infinite-length) extensive-form game with finitely many players.

Definition 10.3.1 (Multi-agent ε -best response and ε -Nash equilibrium)

The policy π_i is an ε -best response (ε , “epsilon”, a small positive tolerance) after history $h_{<t}^i$ iff

$$V_{\sigma_i}^*(h_{<t}^i) - V_{\sigma_i}^{\pi_i}(h_{<t}^i) < \varepsilon$$

If at some time step t , all agents’ policies are ε -best responses, we have an ε -Nash equilibrium.

Plain English. $V_{\sigma_i}^*$ is the best possible value (expected future reward) agent i could achieve in its own **subjective environment** σ_i (defined next section), and $V_{\sigma_i}^{\pi_i}$ is the value it actually achieves by following π_i . If the gap between the two is less than ε for every agent simultaneously, no agent is leaving more than ε worth of value on the table – an approximate, “fuzzy” Nash equilibrium. The property of multi-agent systems analogous to *asymptotic optimality* (Chapters 8–9) is convergence to an ε -Nash equilibrium.

Outline

- 1 10.1 From Preferences to Utilities
- 2 10.2 Game Theory
- 3 10.3 Multi-Agent Extensive-Form Games
- 4 10.4 Strategic Games vs Reinforcement Learning**
- 5 10.5 Reflective Oracles
- 6 10.6 The Grain of Truth
- 7 10.7 Reflective AIXI
- 8 10.8 Exercises
- 9 10.9 History and References

A Terminology Dictionary

There are a lot of immediate similarities between strategic games and the sequential decision theory / (history-based) reinforcement learning setup from earlier chapters. Both have a set of actions an agent can take, and both have some sort of reward or utility that the agent wants to maximize.

Reinforcement learning	=	Game theory
stochastic policy	=	mixed strategy
deterministic policy	=	pure strategy
agent	=	player
multi-agent environment	=	infinite extensive-form game
value	=	payoff/utility
(finite) history	=	history
infinite history	=	path of play
asymptotic optimality	=	convergence to Nash

Table 10.2. Terminology dictionary between reinforcement learning and extensive-form game theory [LTF16].

The Subjective Environment σ_i

Each agent i acts in a **subjective environment** σ_i , obtained by joining the multi-agent environment σ with the policies $\pi_1, \dots, \pi_{i-1}, \pi_{i+1}, \dots, \pi_n$ of everyone *else*, by marginalizing (summing) over the histories that π_i does not see. Together with policy π_i , the environment σ_i yields a distribution over the histories of agent i :

$$\sigma_i^{\pi_i}(h_{<t}^i) := \sum_{h'_{<t}, j \neq i} \sigma^{\pi_{1:n}}(h_{<t})$$

Plain English. We obtain agent i 's subjective, single-agent view of the world by averaging away (marginalizing over) all the possible histories $h'_{<t}$ that the *other* agents $j \neq i$ could have experienced, weighted by how likely each is under the full joint history distribution.

It is crucial to note that the subjective environment σ_i and the policy π_i are *ordinary* (single-agent) environments and policies, so we can use the history-based RL formalism from Chapters 6 and 7. Furthermore, we can interpret policy π_i as a (one-shot) mixed strategy α_i of player/agent i , and the value $V_{\sigma_i}^{\pi_i}(\epsilon)$ as their utility $U_i(\alpha)$ – this reduces the extensive-form game to a normal-form game, so many results for the latter apply here too.

From Strategic Games to Reinforcement Learning

We can convert any extensive-form strategic game into the reinforcement learning setting: let the actions in the game be the actions of the agent, and the utilities $U_i(\alpha)$ be the value $V_{\sigma_i}^{\pi_i}$. The agent represents one of the players, and the environment simulates the actions of all other players.

However simple this conversion sounds, it misses the point of strategic games: modelling the actions of an *intelligent adversary*, or learning to cooperate/compromise so both players benefit. In classical MDP reinforcement learning this is rarely captured, since results typically assume *stationary* environments (the strategy of all other players is fixed) – meaning the opponents cannot learn or adapt when they are being exploited.

This is not to say *all* RL lacks intelligent adversaries: Backgammon [Tes94], Go [SH+16], and StarCraft [VBC+19] are interesting precisely because of the intelligent adversary. Often such environments can be represented as zero-sum games, with the opponent's reward as the negative of the agent's.

Why this matters for AIXI. AIXI assumes the true environment lies within its Bayesian mixture class $\mathcal{M}_{sol}$. This assumption may hold for simple strategic games, but *fails* if AIXI were to play against itself (or another AIXI-like agent) – a reflective problem addressed in the remaining sections of this chapter.

From Reinforcement Learning to Strategic Games

This transformation can also be **reversed**: by keeping the agent's action space as it was in the RL setting, we represent each state or observation of the environment as an action taken by the "opponent". The agent receives a payoff equal to the reward for every action-state pair, and the opponent receives the agent's negative payoff.

A notable limitation. The strategic game's "opponent" may not aim for optimal actions, since the environment selects actions based on its own distribution. This contradicts one of game theory's core principles: that all players strive for optimal performance. Setting the opponent's payoff matrix to identically zero does not help either: *any* best response strategy of the agent to *any* strategy of the opponent is then trivially a Nash equilibrium.

Regardless, various aspects of reinforcement learning can be adapted to the strategic game framework:

- MDPs can be represented by **Markov opponents**.
- Partially observable environments can be mirrored by **partially observable opponent actions**.
- Computable environments can be incorporated by ensuring the game and its opponents are **computable**.

Outline

- 1 10.1 From Preferences to Utilities
- 2 10.2 Game Theory
- 3 10.3 Multi-Agent Extensive-Form Games
- 4 10.4 Strategic Games vs Reinforcement Learning
- 5 10.5 Reflective Oracles**
- 6 10.6 The Grain of Truth
- 7 10.7 Reflective AIXI
- 8 10.8 Exercises
- 9 10.9 History and References

The Self-Modelling Problem

In computational game theory, we can model players as agents running on Turing machines. In this setup, how will two *optimal* agents act if they know the other agent's code, and both know they have the code of the other, etc? Naively, each agent will run the other agent's program, which will run their own program, which will run the other agent's program, and so on – an infinite regress.

One fix is to only run this recursion a fixed number of times – but what if the opponent recurses *more*? To overcome this self-modelling problem, [FTC15] introduced a probabilistic oracle called a **Reflective Oracle**: a truly remarkable construction. When an agent is run with a Reflective Oracle, it is able to reason about itself and overcome the self-modelling problem.

Probabilistic Turing Machines and Semimeasures

Let $\mathcal{T}$ denote the set of **probabilistic Turing machines**. Each $T \in \mathcal{T}$ corresponds to a conditional semimeasure ν_T (“nu sub T ”), where $\nu_T(a \mid x)$ is the probability that T outputs $a \in \mathbb{B}$ given input $x \in \mathbb{B}^*$ ($\mathbb{B}$ = the set of finite/infinite bit strings). Note that $\sum_{a \in \mathbb{B}} \nu_T(a \mid x)$ may be smaller than 1, since T may not halt. We can define $\nu_T(x)$ via the chain rule,

$$\nu_T(x) := \prod_{k=1}^{\ell(x)} \nu_T(x_k \mid x_{<k})$$

which is **lower semicomputable** (we can approximate it from below, getting arbitrarily close but never overshooting, in finite time), since each factor is lower semicomputable. This construction differs from a general solomonoff semimeasure $\nu \in \mathcal{M}_{sol}$, since for a general ν , the predictive distribution $\nu(a \mid x)$ may not be lower semicomputable.

Oracles. An *oracle* is a function which gives access to information that would otherwise be hard or impossible to compute. The oracle receives a question (a *query*) and provides a yes or no answer. A well-known example is the (incomputable) **Halting oracle**: it takes an (encoded) machine-input pair $\langle T, x \rangle$ and returns *Yes* if machine T halts on input x , and *No* otherwise.

Oracle Machines

We extend Turing machines and Turing computability to include oracles. Let T^O denote a Turing machine $T \in \mathcal{T}$ with access to oracle O : that is, T^O is an **oracle machine**, an extended (probabilistic) Turing machine that additionally has an oracle tape and oracle head used to query the oracle and write back 1 = Yes or 0 = No (counted as only one time step) [RJ87]. For instance, if O is the Halting oracle, T^O now has the power to solve the halting problem. Of course, due to the **Church–Turing thesis** (Thesis 2.6.6), no real computer can compute T^O for incomputable oracles O . Let ν_T^O denote the semimeasure induced by T^O .

The oracles $O : \mathcal{T} \times \mathbb{B}^* \times \mathbb{Q} \rightarrow \Delta\mathbb{B}$ we consider take as input (an encoding of) a Turing machine and an input on that Turing machine, as well as a rational number, and return a (possibly randomized) Yes = 1 or No = 0 based on this input.

The specific oracle we consider has the following property: each oracle takes a query $\langle T, x, z \rangle$ – a Turing machine T , an input x to T , and a rational number z – and returns 1 if the probability that T^O outputs 1 on input x is greater than z , that is, for the induced semimeasure $\nu_T^O(1 \mid x) > z$. It returns 0 if $\nu_T^O(0 \mid x) > 1 - z$. If neither is satisfied, the oracle may return 0 or 1 randomly, as it pleases.

Definition 10.5.1: Reflective Oracle

Definition 10.5.1 (Reflective Oracle)

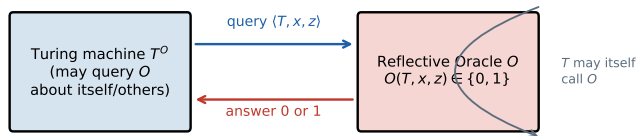
An oracle $O : \mathcal{T} \times \mathbb{B}^* \times \mathbb{Q} \rightarrow \Delta\mathbb{B}$ is *reflective* if and only if for all queries $(T, x, z) \in \mathcal{T} \times \mathbb{B}^* \times \mathbb{Q}$,

- If $\nu_T^O(1 \mid x) > z$ then $O(T, x, z) = 1$,
- If $\nu_T^O(0 \mid x) > 1 - z$ then $O(T, x, z) = 0$,
- Else $O(T, x, z)$ may return 0 or 1 at random (with any probability).

Plain English. The oracle must be *consistent* about itself: since the queried machine T may itself query the oracle O (possibly about itself, or about other machines that query O , and so on), the oracle's answers must be compatible with the very semimeasure they help define – hence the name *reflective*.

Visualizing the Reflective Oracle

The reflective oracle lets a Turing machine T^O query information about a semimeasure – including possibly *about itself* – while guaranteeing the answers stay logically consistent.



The machine T^O sends a query $\langle T, x, z \rangle$; the oracle O answers 0 or 1 following Definition 10.5.1's three cases. Since T may itself contain calls to O , the oracle's answer must remain consistent under this self-reference – the defining challenge that makes reflective oracles remarkable.

Definition 10.5.2 and the Normalization Formula (10.5.3)

Definition 10.5.2 (Reflective-oracle-computable)

A semimeasure is *reflective-oracle-computable* if and only if it is computable on a probabilistic Turing machine with access to a Reflective Oracle. That is, there exists a probabilistic Turing machine T and Reflective Oracle O such that the semimeasure can be computed with T^O .

Recall that *normalizing* a semimeasure ν means increasing it to a measure $\bar{\nu}$ such that $\nu(0 \mid x) + \nu(1 \mid x) = 1$. We can normalize the Reflective Oracle semimeasure ν_T^O for any probabilistic Turing machine T , giving a Reflective Oracle *measure* $\bar{\nu}_T^O$, using binary search with the oracle O to find the *crossover point* $z^* \in [0, 1]$ where $O(T, x, z)$ changes from returning 1 to 0, i.e. $\nu_T^O(1 \mid x) \leq z^* \leq 1 - \nu_T^O(0 \mid x)$. If $\nu_T^O(1 \mid x) + \nu_T^O(0 \mid x) < 1$, z^* may be random, so we take the expectation:

$$\bar{\nu}_T^O(1 \mid x) := \mathbb{E}[z^* \mid x, \nu_T^O] =: 1 - \bar{\nu}_T^O(0 \mid x) \quad \text{for all } x \quad (10.5.3)$$

By construction, $\bar{\nu}_T^O(a \mid x)$ is a reflective-oracle-computable (properly normalized) measure, and $\bar{\nu}_T^O(a \mid x) \geq \nu_T^O(a \mid x)$.

Theorem 10.5.4: Existence of a Reflective Oracle

It is far from obvious whether Reflective Oracles even exist, and if so, whether they are reasonably computable. Remarkably, both questions have the answer: yes.

Theorem 10.5.4 (Existence of a limit-computable Reflective Oracle [LTF16])

There exists a limit-computable (Definition 2.6.13) Reflective Oracle.

Plain English. A **limit-computable** function is one for which there is a computable sequence of guesses that eventually converges to (“settles on”) the true value, even though we may never know *when* it has converged. This is weaker than ordinary computability, but very low in the arithmetic hierarchy (Δ_2^0 , Definition 2.6.17).

The proof requires constructing an infinite hierarchy of partially reflective partial oracles and is beyond the scope of the book [LTF16]. One can also show that Reflective Oracles are *not* Halting Oracles (due to their randomization), and that they are not computable.

Python: Simulating a Reflective Oracle's Decision Rule

We cannot literally compute an uncomputable oracle, but we can simulate Definition 10.5.1's three-way decision rule for a toy machine T whose true output-1 probability $p = \nu_T^O(1 \mid x)$ is known to us (the simulator), for several queried thresholds z .

```
1  import random
2  random.seed(0)
3
4  def reflective_oracle_query(p, z):
5      """p = true P(T outputs 1 | x); z = queried threshold."""
6      if p > z:
7          return 1                # nu_T^O(1|x) > z
8      elif (1 - p) > (1 - z):
9          return 0                # nu_T^O(0|x) > 1-z
10     else:
11         return random.choice([0, 1]) # free choice region
12
13 p = 0.62 # true probability that T outputs 1
14 for z in [0.10, 0.40, 0.62, 0.80, 0.95]:
15     outcome = reflective_oracle_query(p, z)
16     print(f"query z={z:.2f}: O(T,x,z) = {outcome}")
17
18 # query z=0.10: O(T,x,z) = 1      (p=0.62 > z=0.10)
19 # query z=0.40: O(T,x,z) = 1      (p=0.62 > z=0.40)
20 # query z=0.62: O(T,x,z) = 0 or 1 (tie: free-choice region, p==z)
21 # query z=0.80: O(T,x,z) = 0      ((1-p)=0.38 > 1-z=0.20)
22 # query z=0.95: O(T,x,z) = 0      ((1-p)=0.38 > 1-z=0.05)
```

Outline

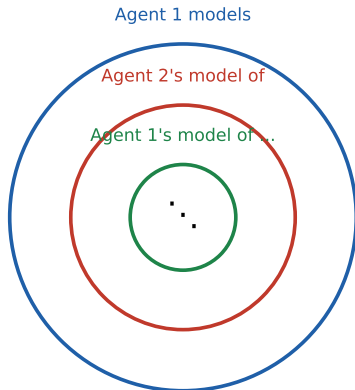
- 1 10.1 From Preferences to Utilities
- 2 10.2 Game Theory
- 3 10.3 Multi-Agent Extensive-Form Games
- 4 10.4 Strategic Games vs Reinforcement Learning
- 5 10.5 Reflective Oracles
- 6 10.6 The Grain of Truth**
- 7 10.7 Reflective AIXI
- 8 10.8 Exercises
- 9 10.9 History and References

The Grain of Truth Problem

When an agent is in a multi-agent setting, and each agent believes that every other agent is acting optimally, what should each agent do? This is the **Grain of Truth** problem. Each agent models the behaviour of the opposing agents, and each of these opposing agents is modelling the behaviour of the agent, which includes models of the opposing agents, and so on. This can go on forever, or up to the maximum depth allowed by each agent.

The Grain of Truth problem is finding *when* this chain of beliefs about opponents' beliefs about itself has a solution. To solve it, we take a **Bayesian** approach.

Visualizing the Infinite Regress



Agent 1 models the environment, which includes Agent 2. Agent 2's model of Agent 1 in turn contains a model of Agent 1's model of Agent 2, and so on – an infinite nested regress of “models of models”. The Grain of Truth problem asks when this infinite chain is well defined and contains a grain (a small but nonzero piece) of the truth needed for learning to work

Strategy: Solving Grain of Truth with Reflective Oracles

In this section we show how Reflective Oracles can be used to solve the Grain of Truth problem, in three steps:

- 1 Define a class of environments $\mathcal{M}$.
- 2 Show that the Bayesian mixture over $\mathcal{M}$ is itself contained *within* the class $\mathcal{M}$.
- 3 Show that the optimal policy over this class is Reflective Oracle computable.

Formally, the Grain of Truth problem entails identifying a collection of environments $\mathcal{M}$ such that the optimal policy $\pi_{\xi_{\mathcal{M}}}^*$ derived from the Bayesian mixture $\xi_{\mathcal{M}}$ over $\mathcal{M}$ itself belongs to $\mathcal{M}$. It is crucial that $\mathcal{M}$ represents a class encompassing *both* policies and environments simultaneously, since we are examining a multi-agent setting where the environment is also considered an (opponent) policy.

Reflective Oracles have so far only been developed for binary sequences. This is not a problem, because we can bijectively map actions and observations to binary complete prefix codes [MH21b, CHV22a, MH21a], and take products of conditional binary predictive distributions, which we henceforth assume.

Definition 10.6.1: Reflective Environments

Definition 10.6.1 (Reflective environments)

The class of *reflective environments* with respect to a set of Turing machines $\mathcal{T}$ and a fixed reflective oracle O is defined by

$$\mathcal{M}_r^O := \{\bar{\nu}_T^O : T \in \mathcal{T}\} \quad (\text{see (10.5.3)})$$

Recall the definition of a Bayesian mixture (Definitions 7.2.1 and 7.2.2), this time over $\mathcal{M}_r^O$:

$$\xi(e_t \mid h_{<t}a_t) := \sum_{\nu \in \mathcal{M}_r^O} w(\nu \mid h_{<t}) \nu(e_t \mid h_{<t}a_t)$$

but with one key difference: $w(\nu \mid h_{<t})$ is the recursively defined *renormalized posterior*,

$$w(\nu \mid h_{1:t}) := w(\nu \mid h_{<t}) \frac{\nu(e_t \mid h_{<t}a_t)}{\bar{\xi}(e_t \mid h_{<t}a_t)}$$

We can choose any lower semicomputable prior, e.g. $w_{\bar{\nu}_T^O} = 2^{-K(T)}$, in line with AIXI, where K is the Kolmogorov complexity (Section 2.7).

Theorem 10.6.2: Bayes Is a Reflective Environment

Theorem 10.6.2 (Bayes is a reflective environment [Lei16b])

The Bayesian normalized mixture $\bar{\xi}$ is an element of $\mathcal{M}_r^O$, that is, $\bar{\xi} \in \mathcal{M}_r^O$.

Why this matters. This is the crucial step: it says the Bayes-optimal environment (the “average” over everything the agent currently believes) is *itself* a valid member of the very class $\mathcal{M}_r^O$ it was built from – exactly the self-consistency the Grain of Truth problem demands.

Proof sketch. By induction over t , it is easy to see that ξ is reflective-oracle-lower-semicomputable. Assume $w(\nu \mid h_{<t})$ is reflective-oracle-lower-semicomputable, which is true for $t = 1$. This implies $\xi(e_t \mid h_{<t}a_t)$ is reflective-oracle-lower-semicomputable. Hence there exists an oracle machine T' such that $\xi(e_t \mid h_{<t}a_t) = \nu_{T'}^O(e_t \mid h_{<t}a_t)$, hence the normalized $\bar{\xi}(e_t \mid h_{<t}a_t) = \bar{\nu}_{T'}^O(e_t \mid h_{<t}a_t)$ is reflective-oracle-computable. This normalization is the key difference from Definition 7.2.2, and makes the posterior $w(\nu \mid h_{1:t})$ reflective-oracle-lower-semicomputable, completing the inductive step. Since $T' \in \mathcal{T}$, we have $\bar{\nu}_{T'}^O \in \mathcal{M}_r^O$. ■

Theorem 10.6.3: Optimal Policies Are Reflective-Oracle-Computable I

Theorem 10.6.3 (Optimal policies are reflective-oracle-computable [Lei16b])

For every $\nu \in \mathcal{M}_r^O$ there is a ν -optimal (stochastic) policy π_ν^* . If discount function γ_k and normalizer Γ_k are computable, then π_ν^* is reflective-oracle-computable.

Note that even though deterministic optimal policies always exist, those policies are typically *not* reflective-oracle-computable.

Theorem 10.6.3: Optimal Policies Are Reflective-Oracle-Computable II

Proof sketch. Recall we can write the optimal value function (Lemma 6.6.3) as

$$V_{\nu}^{*,m}(h_{<t}) = \frac{1}{\Gamma_t} \max_{a_t \in \mathcal{A}} \sum_{e_t \in \mathcal{E}} \cdots \max_{a_m \in \mathcal{A}} \sum_{e_m \in \mathcal{E}} \prod_{k=t}^m \nu(e_k \mid h_{<k} a_k) \sum_{k=t}^m \gamma_k r_k$$

Since every component of the value function is computable or reflective-oracle-computable, and there are only finitely many terms, the quantity itself is reflective-oracle-computable, as the operations combining the elements are computable operations. As $V_{\nu}^*(h_{<t}) := \lim_{m \rightarrow \infty} V_{\nu}^{*,m}(h_{<t})$, the value function is monotone increasing in m (rewards are bounded in $[0, 1]$), and the tail beyond time $m + 1$ is bounded by Γ_{m+1} , which is computable and converges to 0 as $m \rightarrow \infty$. So for any desired $\varepsilon > 0$ we can choose m such that $\Gamma_{m+1} < \varepsilon/2$ and compute V_{ν}^* to any desired accuracy, hence it is reflective-oracle-computable.

Theorem 10.6.3: Optimal Policies Are Reflective-Oracle-Computable III

Restricting to only two actions, $\alpha = 0$ and $\beta = 1$: since V_ν^* is reflective-oracle-computable, there exists a probabilistic Turing machine T such that

$$\nu_T^O(1 \mid h_{<t}) := \frac{1}{2} [V_\nu^*(h_{<t}\alpha) - V_\nu^*(h_{<t}\beta) + 1] =: \nu_T^O(0 \mid h_{<t})$$

Then we can define a policy π as

$$\pi(h_{<t}) := \begin{cases} \alpha & \text{if } O(T, h_{<t}, \frac{1}{2}) = 1 \\ \beta & \text{if } O(T, h_{<t}, \frac{1}{2}) = 0 \end{cases}$$

Plain English. The oracle machine converts the *difference* in value between taking action α versus β into a probability centred on $\frac{1}{2}$; querying the reflective oracle at threshold $z = \frac{1}{2}$ then picks out whichever action has strictly higher value. If $V_\nu^*(h_{<t}\alpha) > V_\nu^*(h_{<t}\beta)$, then $\nu_T^O(1 \mid h_{<t}) > \frac{1}{2}$, so $O(T, h_{<t}, \frac{1}{2}) = 1$ and π takes the optimal action α ; symmetrically if β is strictly better. If the two actions tie, both are optimal, so π is optimal whatever the oracle happens to return. Therefore π is ν -optimal and reflective-oracle-computable. ■

Theorem 10.6.4: Solution to the Grain of Truth Problem

Combining the previous two theorems gives us the solution to the Grain of Truth problem.

Theorem 10.6.4 (Solution to the Grain of Truth problem [Lei16b])

For every lower semicomputable prior $w \in \Delta' \mathcal{M}_r^O$ and computable γ and Γ , the Bayes-optimal ($\bar{\xi}$ -optimal) policy $\pi_{\bar{\xi}}^*$ is reflective-oracle-computable, where $\bar{\xi}$ is the normalized Bayes-mixture defined above.

Proof. This immediately follows from Theorems 10.6.2 and 10.6.3. ■

This solves a fundamental problem that had been open 20+ years for the first non-trivial class of environments – and indeed arguably for the most interesting class, which includes *all* computable environments, while also not being too crazy: every $\nu \in \mathcal{M}_r^O$ is at least limit-computable. A **diagonalization argument** prevents a Grain of Truth in $\mathcal{M}_{comp}$ (all computable environments) and $\mathcal{M}_{sol}$ (all lower-semicomputable semimeasures), so in a sense $\mathcal{M}_r^O$ is the best solution one can hope for.

Outline

- 1 10.1 From Preferences to Utilities
- 2 10.2 Game Theory
- 3 10.3 Multi-Agent Extensive-Form Games
- 4 10.4 Strategic Games vs Reinforcement Learning
- 5 10.5 Reflective Oracles
- 6 10.6 The Grain of Truth
- 7 10.7 Reflective AIXI**
- 8 10.8 Exercises
- 9 10.9 History and References

Informed Reflective Agents

The solution of the Grain of Truth problem allows us to generalize the asymptotic optimality results of Chapter 9 to variants of AIXI in the multi-agent setting.

Let σ be a multi-agent environment (Section 10.3), and let $\pi_{\sigma_1}^*, \dots, \pi_{\sigma_n}^*$ be such that for each i the policy $\pi_{\sigma_i}^*$ is an optimal policy in agent i 's subjective environment σ_i . At first glance this seems

ill-defined: the subjective environment σ_i depends on each other policy $\pi_{\sigma_j}^*$ for $j \neq i$, which depends on the subjective environment σ_j , which in turn depends on the policy $\pi_{\sigma_i}^*$! However, this circular definition actually has a well-defined solution.

Theorem 10.7.1 (Optimal multi-agent policies)

For any reflective-oracle-computable multi-agent environment σ , the optimal policies $\pi_{\sigma_1}^*, \dots, \pi_{\sigma_n}^*$ exist and are reflective-oracle-computable.

Subgame Perfect Equilibrium and Computable Environments

Note the strength of Theorem 10.7.1: each of the policies $\pi_{\sigma_i}^*$ is acting optimally *given the knowledge of everyone else's policies*. Hence optimal policies play 0-best responses by definition, so if every agent is playing an optimal policy, we have a **Nash equilibrium**. Moreover, each agent also acts optimally on the *counterfactual* histories that do *not* end up being played. This stronger property is called a **subgame perfect** Nash equilibrium.

In other words, Theorem 10.7.1 states the existence and reflective-oracle-computability of a subgame perfect Nash equilibrium in any reflective-oracle-computable multi-agent environment. From Theorem 10.5.4 we then get that these subgame perfect Nash equilibria are limit-computable.

Corollary 10.7.2 (Solution to computable multi-agent environments)

For any *computable* multi-agent environment σ , the optimal policies $\pi_{\sigma_1}^*, \dots, \pi_{\sigma_n}^*$ exist and are limit-computable.

Learning Reflective Agents

Since our class $\mathcal{M}_r^O$ solves the grain of truth problem, the famous result by Kalai and Lehrer [KL93] immediately implies that for any Bayesian agents $\pi_1, \dots, \pi_n$ interacting in an infinitely repeated game, and for all $\varepsilon > 0$ and all $i \in \{1, \dots, n\}$, there is almost surely a $t_0 \in \mathbb{N}$ such that for all $t \geq t_0$ the policy π_i is an ε -best response.

But there is an important catch: this hinges on every agent knowing the game, and knowing that all other agents are Bayesian. Otherwise convergence to an ε -Nash equilibrium may **fail**, as illustrated by the following example, built around a **dogmatic prior** (Section 8.2.1).

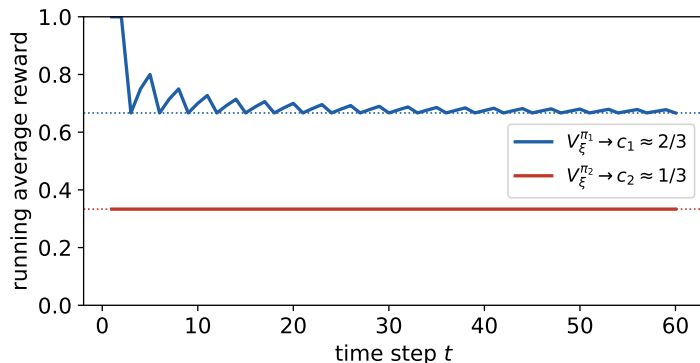
A dogmatic prior assigns very high probability to “going to hell” (reward 0 forever) if the agent deviates from a given computable policy π . For a Bayesian agent, it is thus only worth deviating from π if the agent thinks the prospects of following π are already very poor. This implies that, for general multi-agent environments and without additional assumptions on the prior, we cannot prove any meaningful convergence result about Bayesian agents acting in an unknown multi-agent environment.

Example 10.7.3: Reflective Bayesians Playing Matching Pennies

Consider the game of **matching pennies** (Section 10.2.3). Let $\mathcal{E} = \{0, 1\}$ be the set of rewards (observations are vacuous), and define the multi-agent environment σ to give reward 1 to agent 1 iff $a_t^1 = a_t^2$ (0 otherwise), and reward 1 to agent 2 iff $a_t^1 \neq a_t^2$ (0 otherwise). Neither agent knows *a priori* that they are playing matching pennies, nor that they are playing an infinitely repeated game with one other player.

Let π_1 be the policy that takes the action sequence $(HHT)^\infty$ (repeat Heads, Heads, Tails forever), and let $\pi_2 := \pi_H$ be the policy that always takes action H . The average reward of policy π_1 is $2/3$ and the average reward of policy π_2 is $1/3$. Let ξ be a universal mixture (Definition 7.2.2). By Theorem 7.3.1, $V_\xi^{\pi_1} \rightarrow c_1 \approx 2/3$ and $V_\xi^{\pi_2} \rightarrow c_2 \approx 1/3$ almost surely when following policies (π_1, π_2) .

Example 10.7.3: The Values Lock In



Both agents' running average rewards converge to fixed constants: $V_{\xi}^{\pi_1} \rightarrow c_1 \approx 2/3$ (blue) and $V_{\xi}^{\pi_2} \rightarrow c_2 \approx 1/3$ (red). By Theorem 8.2.1, dogmatic mixtures ξ'_1, ξ'_2 then exist under which each agent's Bayes-optimal policy *always* sticks to π_1 , respectively π_2 – since deviating looks catastrophic under a dogmatic prior.

Example 10.7.3, Continued: Why This Is Not Nash

Since $V_{\xi}^{\pi_1} \rightarrow c_1 \approx 2/3 > 0$ and $V_{\xi}^{\pi_2} \rightarrow c_2 \approx 1/3 > 0$, there is an $\varepsilon > 0$ with $V_{\xi}^{\pi_1} > \varepsilon$ and $V_{\xi}^{\pi_2} > \varepsilon$ for all time steps – both agents are, in this weak sense, doing fine. Yet:

This does not converge to a (approximate) Nash equilibrium. Both agents' values have converged and stabilized (they are locked onto fixed constants $c_1 \approx 2/3$ and $c_2 \approx 1/3$), yet neither agent is actually best-responding: Agent 2 knows (or could learn) that Agent 1 always plays $(HHT)^{\infty}$, so Agent 2 could switch to always playing the *opposite* of Agent 1's predicted action and get reward 1 every single time, instead of $1/3$. Likewise, Agent 1 could switch to always playing H (matching Agent 2's constant action) and get reward 1 every time, instead of $2/3$. Because each agent's dogmatic prior makes deviating from its own fixed policy look catastrophic (a “go to hell” event), *neither ever deviates*, and the pair is permanently stuck away from any ε -Nash equilibrium.

Python: Simulating the Dogmatic-Prior Matching-Pennies Example

We simulate $\pi_1 = (HHT)^\infty$ against $\pi_2 = \pi_H$ (always Heads), compute their running-average rewards, and compare against the reward each agent *could* get by best-responding instead.

```
1  T = 30
2  pi1 = [1, 1, 0] * (T // 3)      # H=1,H=1,T=0 repeating
3  pi2 = [1] * T                    # always Heads (H=1)
4  r1 = [1 if pi1[t] == pi2[t] else 0 for t in range(T)] # agent 1: reward if match
5  r2 = [1 if pi1[t] != pi2[t] else 0 for t in range(T)] # agent 2: reward if mismatch
6  avg1, avg2 = sum(r1) / T, sum(r2) / T
7  print(f"avg reward pi_1 (locked-in) = {avg1:.3f} (theory: 2/3 = 0.667)")
8  print(f"avg reward pi_2 (locked-in) = {avg2:.3f} (theory: 1/3 = 0.333)")
9  # Best response: agent 2 predicts pi_1 and always plays the OPPOSITE action.
10 best_r2 = [1 if pi1[t] != (1 - pi1[t]) else 0 for t in range(T)]
11 print(f"avg reward if agent 2 best-responds = {sum(best_r2)/T:.3f} (vs 0.333)")
12 # Best response: agent 1 predicts pi_2 == always H, and matches it.
13 best_r1 = [1 if 1 == pi2[t] else 0 for t in range(T)]
14 print(f"avg reward if agent 1 best-responds = {sum(best_r1)/T:.3f} (vs 0.667)")
15 # avg reward pi_1 (locked-in) = 0.667 (theory: 2/3 = 0.667)
16 # avg reward pi_2 (locked-in) = 0.333 (theory: 1/3 = 0.333)
17 # avg reward if agent 2 best-responds = 1.000 (vs 0.333)
18 # avg reward if agent 1 best-responds = 1.000 (vs 0.667)
```

Both agents leave a huge amount of value (0.667 and 1 respectively) on the table forever – exactly the failure Example 10.7.3 describes.

Theorem 10.7.4: Convergence to Equilibrium

The following theorem is the main positive convergence result. It states that we *do* get convergence to ε -Nash equilibria in any reflective-oracle-computable multi-agent environment, provided the agents are asymptotically optimal *in mean* (a weaker requirement than the dogmatic agents above satisfy).

Theorem 10.7.4 (Convergence to equilibrium)

Let σ be a reflective-oracle-computable multi-agent environment and let $\pi_1, \dots, \pi_n$ be reflective-oracle-computable policies that are asymptotically optimal in mean (Definition 8.1.6) in the class $\mathcal{M}_r^O$. Then for all $\varepsilon > 0$ and all $i \in \{1, \dots, n\}$, the $\sigma^{\pi_{1:n}}$ -probability that the policy π_i is an ε -best response converges to 1 as $t \rightarrow \infty$.

Contrast with Theorem 10.7.1. In contrast to Theorem 10.7.1, which yields policies that play a *subgame perfect* equilibrium, this is not the case here: the agents typically do not learn to predict off-policy, and thus will generally not play ε -best responses on the counterfactual histories they never see. This weaker form of equilibrium is unavoidable if the agents do not know the environment, because it is impossible to learn the parts of it they do not interact with.

Corollary 10.7.5: Thompson-Sampling Reflective AIXI

Together with Theorem 10.5.4 and the asymptotic optimality of the Thompson sampling policy (Theorem 9.2.1), which is reflective-oracle-computable, we get the following corollary.

Corollary 10.7.5 (Thompson-Sampling reflective AIXI converges to equilibrium)

There are limit-computable policies $\pi_1, \dots, \pi_n$ such that for any computable multi-agent environment σ and for all $\varepsilon > 0$ and all $i \in \{1, \dots, n\}$, the $\sigma^{\pi_{1:n}}$ -probability that the policy π_i is an ε -best response converges to 1 as $t \rightarrow \infty$. Thompson-Sampling AIXI π_{TS} (Algorithm 9.3) with class $\mathcal{M}_r^O$ is such a policy.

Plain English. The Thompson-sampling agents $\pi_{TS,i}$ are allowed to differ from each other: they can use different discount functions, resample at different time steps, and converge at different speeds. All of the Thompson-sampling agents eventually “calm down” and settle on some posterior belief.

This is a truly remarkable solution to the Grain of Truth problem: the *only* assumption on the environment σ is computability, while reflective AIXI is still merely limit-computable (like single-agent AIXI based on $\mathcal{M}_{sol}$) – not practical, but potentially inspiring practical versions of this construction. Reflective oracles have already seen applications in one-shot games [FTC15].

Outline

- 1 10.1 From Preferences to Utilities
- 2 10.2 Game Theory
- 3 10.3 Multi-Agent Extensive-Form Games
- 4 10.4 Strategic Games vs Reinforcement Learning
- 5 10.5 Reflective Oracles
- 6 10.6 The Grain of Truth
- 7 10.7 Reflective AIXI
- 8 10.8 Exercises**
- 9 10.9 History and References

Exercises I

1. [C18] Prove Theorem 10.2.4.
2. [C28m] Prove Theorem 10.2.7.
3. [C20] Prove Theorem 10.2.11.
4. [C25] Prove that specific properties (Markov, computability, stochasticity, partial observability, optimality conditions) carry over in the strategic games to RL conversion (and vice versa).
5. [C28] Prove that the environment class of repeated strategic games admits self-optimizing policies, and hence that π_{ξ}^* is self-optimizing over the environment class of repeated strategic games.
6. [C18] Derive some simple (trivial) environment classes where there is a Grain of Truth.
7. [C25] Extend the proof of Theorem 10.6.3 to an arbitrary finite action space.

Reading the difficulty codes. The bracketed code (e.g. [C18], [C28m]) is the book's difficulty rating: roughly, the number gives an estimated difficulty out of 50, "C" marks a *creative*/research-flavoured exercise, and a trailing letter (e.g. "m") marks additional notes on the exercise's character (such as requiring extra mathematical machinery).

Outline

- 1 10.1 From Preferences to Utilities
- 2 10.2 Game Theory
- 3 10.3 Multi-Agent Extensive-Form Games
- 4 10.4 Strategic Games vs Reinforcement Learning
- 5 10.5 Reflective Oracles
- 6 10.6 The Grain of Truth
- 7 10.7 Reflective AIXI
- 8 10.8 Exercises
- 9 10.9 History and References

History and References I

Game theory. The topic of game theory is well developed in the textbook [OR94]. It is a theoretical framework originating from the work of John von Neumann and Oskar Morgenstern in the 1940s, allowing for a theoretical understanding of strategic interaction between agents in both competitive and cooperative environments [VNM47]. Central to game theory is the concept of **Nash Equilibrium**, introduced by John Nash in the early 1950s, which describes a state in a strategic game where no player can benefit from unilaterally changing strategies while the other players keep their strategies unchanged [NJ50]. This equilibrium is stable, but it may be Pareto dominated by another strategy if players can collude to select it.

This discovery can explain many counter-intuitive results: (1) why adding a new road to an existing road network can cause the travel time to paradoxically increase for all drivers [Bra68]; (2) why deliberately limiting the set of available actions via credible pre-commitment to a strategy can be advantageous [Sch80]. It also has applications explaining strategies that evolve naturally in the animal kingdom: (3) either passive or aggressive strategies can be stable in the animal kingdom [SP73]; (4) animals often display costly signals, like the peacock's elaborate tail, to help find a mate [Ham64]; (5) altruistic behaviours can evolve in a survival-of-the-fittest framework.

History and References II

Mixed strategies and further reading. Often, a deterministic strategy may leave the agent open to being exploited. Mixed strategies allow agents to choose a stochastic strategy to avoid exploitative responses [Neu28]. Besides Nash equilibrium, there are other concepts such as altruistic equilibrium and dominant strategy equilibrium, and Pareto optimality. A complete classification of 2×2 symmetric games can be found in [BWEH22], which covers PD, SH, Chicken, and BoS, but not MP, since the latter is not symmetric.

RL and game theory. The connection between reinforcement learning and game theory is presented in [SLB08], which includes a survey of MDPs for multiplayer games. Another useful resource on this connection is [Cri17], which reviews the multi-agent and multi-objective reinforcement learning literatures. POTMMCP [SKH23] is a theoretically sound multi-agent online-learning MCTS-based planning algorithm with good practical performance, even for problems requiring a large planning horizon. [MJS19] considers deterministic games with local Nash equilibria (non-zero second derivative of gradient dynamics) and shows that gradient descent can get attracted to non-Nash local extrema (or saddle points).

Grain of Truth and reflective oracles. Finding a grain of truth is a famously hard problem [KL93], with many impossibility results [Nac97, FY01, Nac05]. The remarkable solution presented in this chapter is from [LTF16, Lei16b], using previous reflective oracle work from [FST15, FTC15]. Sections 10.3 and 10.5 to 10.7 were taken with permission from [LTF16, Lei16b], with some minimal modification.